

ALGORITHMS II — END-OF-TERM PROJECT

Shortest-Path Queries on Weighted Graphs

Comparative Algorithm Design and Complexity Analysis

Four algorithmic approaches • three graph representations • theory + experiment

Project Group: Berfin Güneri – Murat Emre Öztürk – Berkay Bıyık – Faruk Emre Afşar | Spring Semester 2025–2026

Results: profile "full", seed 20252

The Problem: Shortest-Path Query

- ▶ Given: a weighted directed graph $G = (V, E)$ and weights $w: E \rightarrow \mathbb{R}$
- ▶ Wanted: the minimum total-weight path between two nodes
 - Single-source (SSSP): from one source to all
 - All-pairs (APSP): between every pair of nodes
- ▶ The goal is NOT merely to find the correct answer —
 - but to solve the same problem with different algorithmic decisions
 - and to understand the performance / scaling / memory effects
- ▶ "Query" = the cost of running from one source given a built graph

Takeaway

- There is no single "best algorithm."
- The right choice depends on input size, density, query count, and the sign of the weights.
- This presentation shows that decision with data.

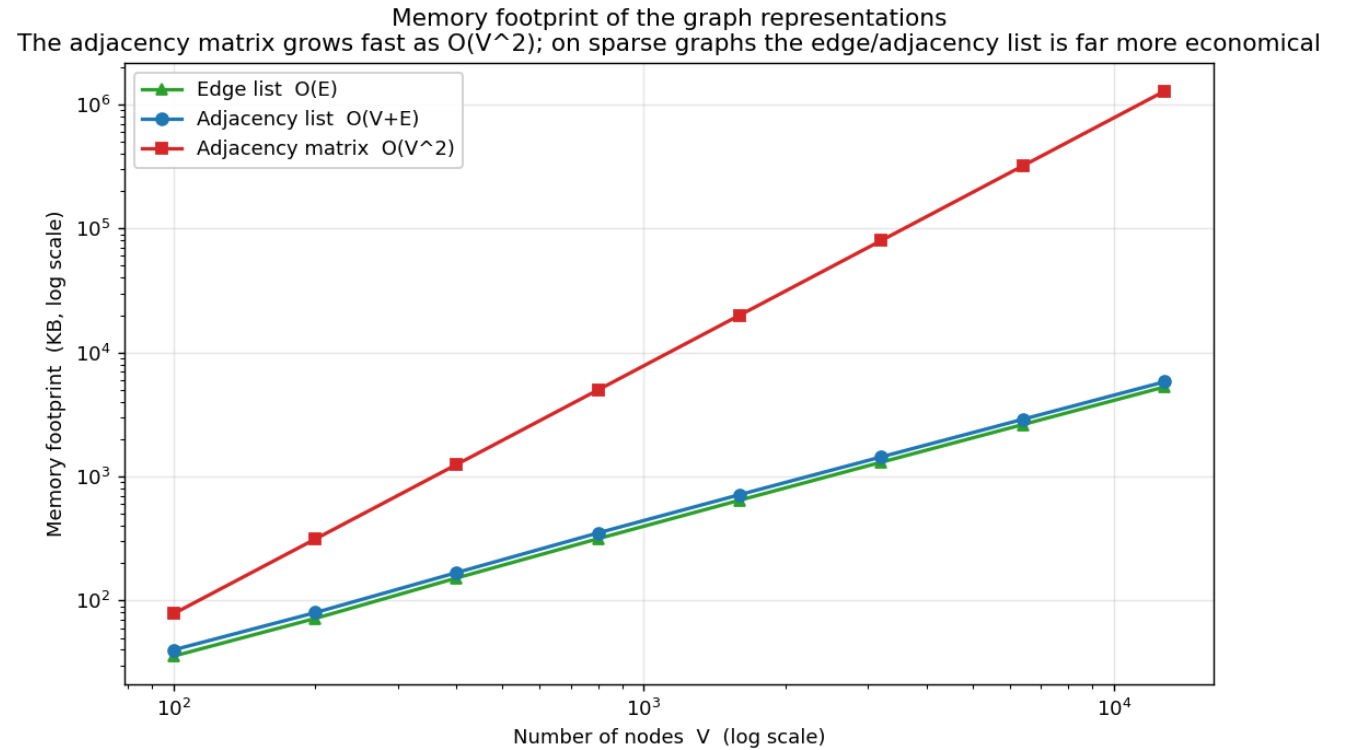
Four Approaches — The Design Space

#	Approach	Representation	Philosophy
1	Dijkstra + Binary Heap	Adjacency list	On-demand, time-optimized
2	Dijkstra + Array Scan	Adjacency matrix	Dense-graph-optimized, simple
3	Bellman-Ford	Edge list	Generality: negative weights
4	Floyd-Warshall	Adjacency matrix	Heavy preprocessing, $O(1)$ query

- ▶ All three graph representations + four distinct strategies are covered
- ▶ Meaningful pairwise comparisons: 1—2 (same algorithm, different data structure) · 1—3 (different strategy) · 1—4 (on-demand vs. preprocessing)
- ▶ A natural split for a four-member group: one approach per member

Graph Representations — A Decision Independent of the Algorithm

- ▶ **Edge list** — $O(E)$ memory; "iterate over all edges" is natural (Bellman-Ford)
- ▶ **Adjacency list** — $O(V+E)$; "the neighbors of u " is fast (Dijkstra-Heap)
- ▶ **Adjacency matrix** — $O(V^2)$; $O(1)$ edge query, vectorizes (Dijkstra-Array, Floyd-Warshall)
- ▶ The representation choice determines both memory and the cost of the fundamental operation



Takeaway

- The adjacency matrix grows as $O(V^2)$: on large sparse graphs, matrix-based approaches DO NOT FIT in memory (Experiment D).

Approach 1: Dijkstra + Binary Heap + Adjacency List

- ▶ Greedy: at each step pick the not-yet-finalized node closest to the source
- ▶ The binary heap answers the "minimum" query in $O(\log V)$
- ▶ "Lazy deletion": an improved node is re-pushed onto the heap
- ▶ Early stopping: stop when the target is reached (speeds up single-pair queries)

Complexity

Time: $O((V + E) \log V)$

Memory: $O(V + E)$

On a sparse graph $\approx O(V \log V)$

Philosophy

On-demand, time-optimized. With negative edges the greedy assumption breaks → it becomes incorrect.

Approach 2: Dijkstra + Array Scan + Adjacency Matrix

- ▶ The same greedy strategy, but with NO priority queue
- ▶ At each step the min node is found by a linear scan of the array, $O(V)$
- ▶ Relaxation is done over a row of the adjacency matrix, $O(V)$
- ▶ The cost is entirely independent of the number of edges

Complexity

Time: $O(V^2)$

Memory: $O(V^2)$

On a dense graph $O(V^2) \leq O(E \log V)$

Philosophy

Structural simplicity, dense-graph-optimized. On large sparse graphs $O(V^2)$ memory $\rightarrow$ infeasible.

Approach 3: Bellman-Ford + Edge List

- ▶ No greedy assumption needed: relax all edges for $V-1$ passes
- ▶ Works CORRECTLY with negative weights
- ▶ If one extra pass still relaxes $\rightarrow$ negative cycle
- ▶ Early stopping: stop if nothing changes (much faster in practice)

Complexity

Time: $O(V \cdot E)$

Memory: $O(V + E)$

Worst case is worse than Dijkstra

Philosophy

Generality — correctness before speed. Solves what Dijkstra cannot.

Approach 4: Floyd-Warshall + Adjacency Matrix

- ▶ Shortest paths between all node pairs (APSP)
- ▶ Dynamic programming: for each intermediate node k , all (i,j) are updated
- ▶ After preprocessing, each query is an $O(1)$ matrix lookup
- ▶ Detects a negative cycle as a negative value on the diagonal

Complexity

Preprocessing: $O(V^3)$

Query: $O(1)$

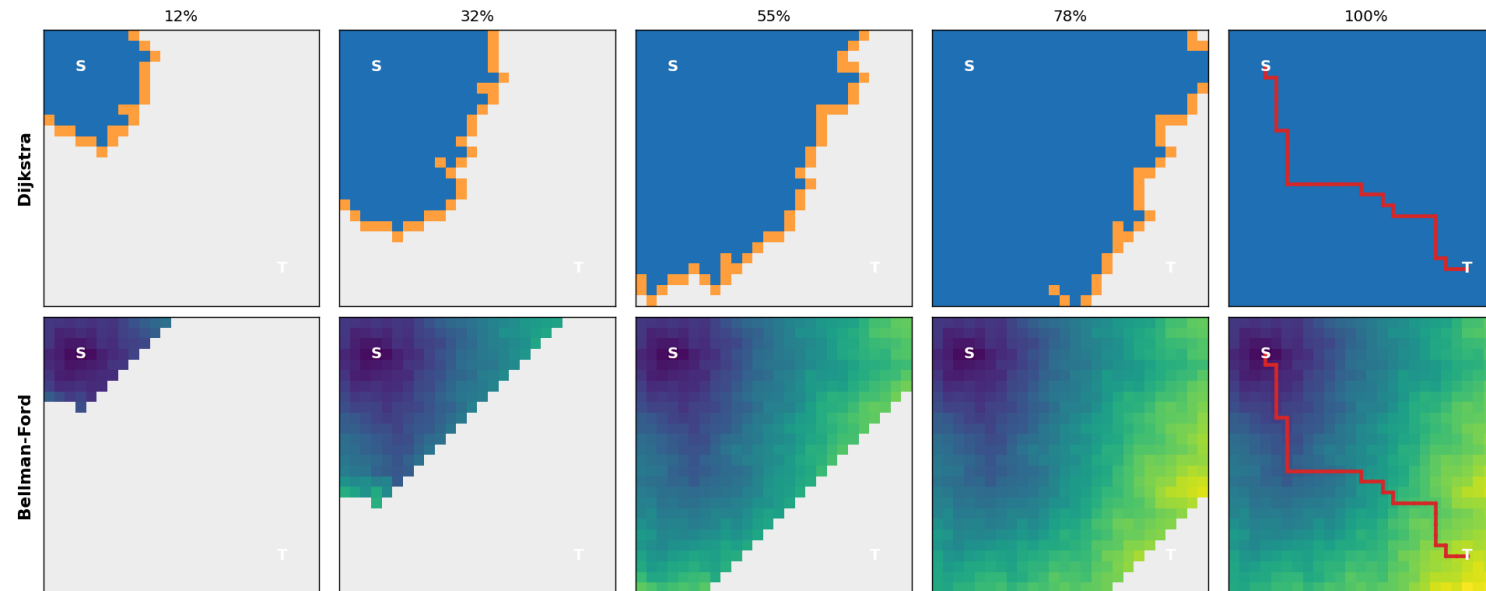
Memory: $O(V^2)$

Philosophy

Heavy preprocessing, ~free queries. Wasteful for a single query, ideal for many.

Algorithms at Work — Visual Simulation

Grid exploration: Dijkstra advances selectively, Bellman-Ford scans all edges (S=source, T=target, red=shortest path)



Takeaway

- Dijkstra (top): the blue region grows in order of distance; it works only on the thin orange "frontier"
- Bellman-Ford (bottom): each pass relaxes all edges INDISCRIMINATELY; the distance field spreads like a wave
- Both find the same path (red) — the difference is IN THE WORK DONE
- The full animations (GIF) are in the results/animations/ folder; can be played live during the presentation

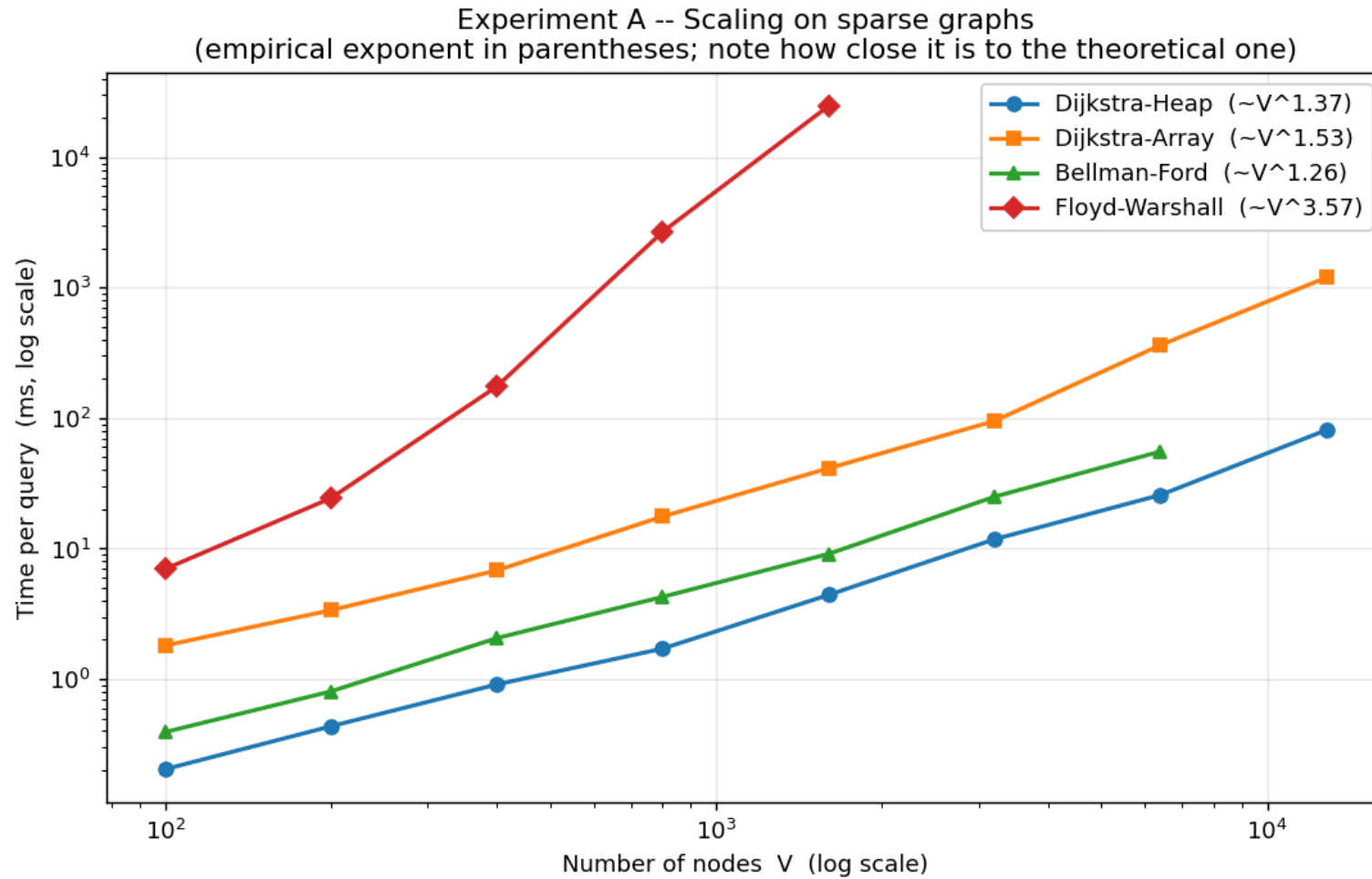
Theoretical Complexity — Summary

Approach	Preprocessing	Query	Memory	Negative weights
Dijkstra + Binary Heap	—	$O((V+E) \log V)$	$O(V+E)$	No
Dijkstra + Array Scan	—	$O(V^2)$	$O(V^2)$	No
Bellman-Ford	—	$O(V \cdot E)$	$O(V+E)$	Yes (+ cycle)
Floyd-Warshall	$O(V^3)$	$O(1)$ lookup	$O(V^2)$	Yes (+ cycle)

Takeaway

- Dijkstra-Heap scales best on sparse graphs · Dijkstra-Array wins on dense graphs ($O(V^2) \leq O(E \log V)$)
- Floyd-Warshall: terrible for a single query, ideal for many — the preprocessing philosophy

Experiment A — Scaling with the Number of Nodes



Takeaway

- The curve ordering exactly confirms the theoretical complexity
- At $V = 1600$, Floyd-Warshall is $\sim 5,610\times$ slower than Dijkstra-Heap
- With $O(V^3)$, Floyd-Warshall becomes "infeasible" earliest
- Dijkstra-Heap is the best-scaling approach

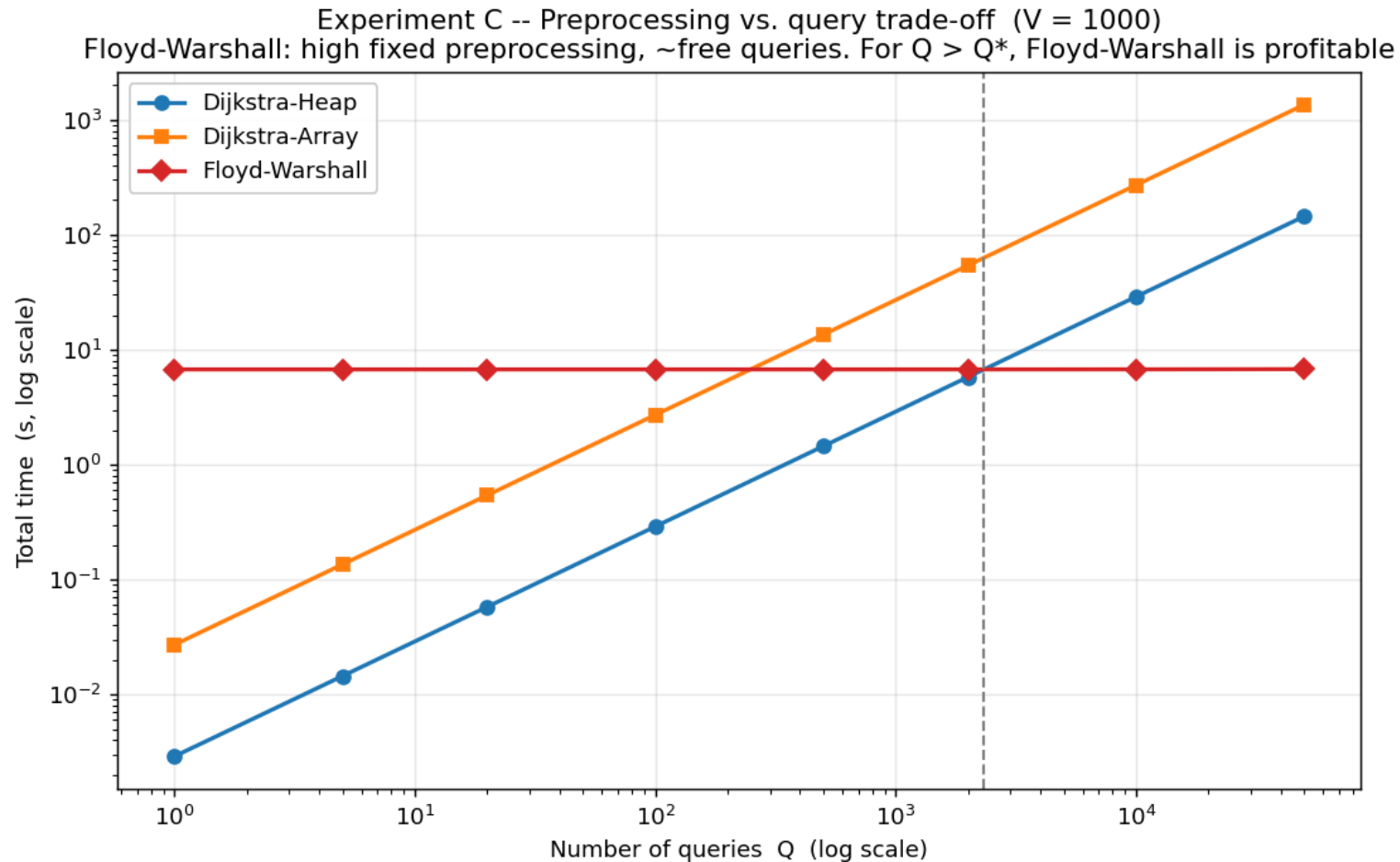
Experiment B — The Effect of Edge Density



Takeaway

- Dijkstra-Array and Floyd-Warshall: time is flat (independent of density)
- Dijkstra-Heap and Bellman-Ford: rise with density
- At the sparse end Dijkstra-Heap leads; as it gets denser Dijkstra-Array overtakes it
- The "best algorithm" depends on the graph's density

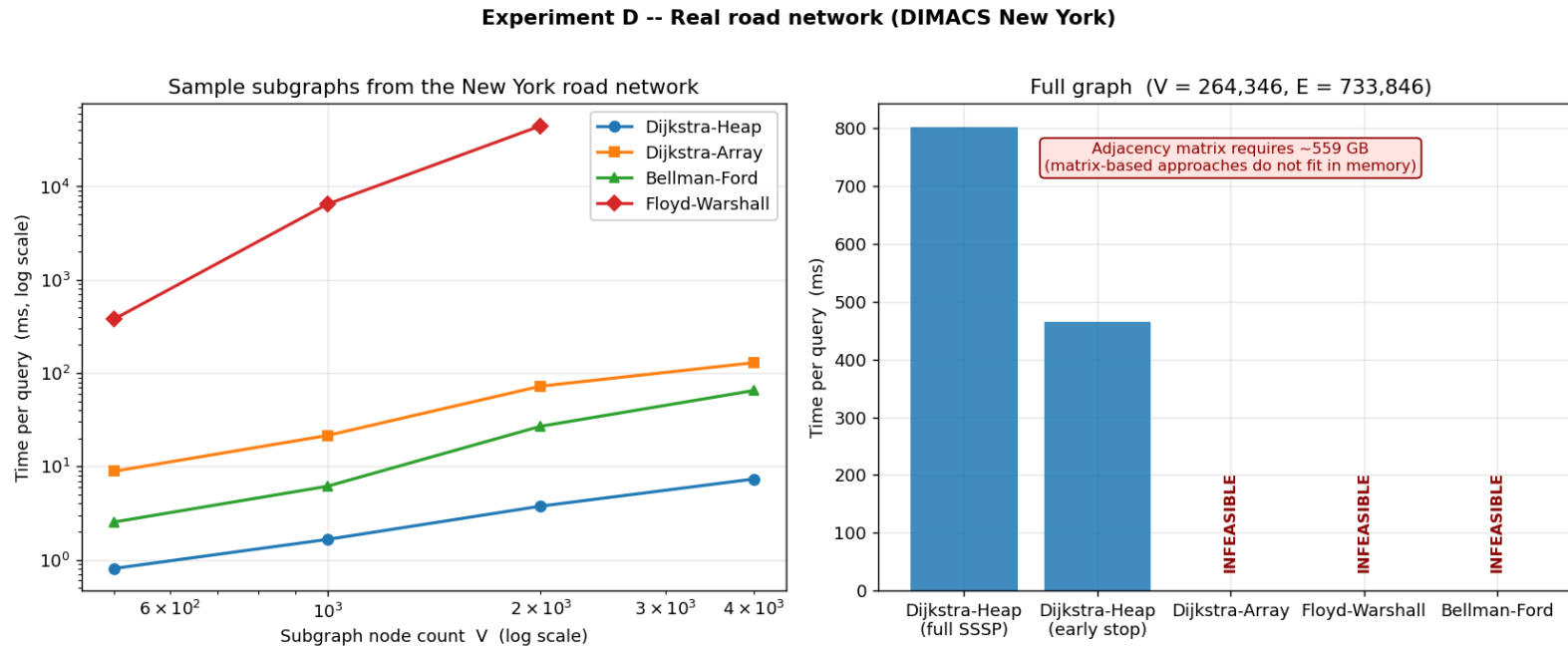
Experiment C — Preprocessing / Query-Count Trade-off



Takeaway

- Dijkstra: pays from scratch on every query (directly proportional)
- Floyd-Warshall: high fixed preprocessing, ~free queries
- Break-even point: $Q^* \approx 2330$ queries
- Heavy preprocessing makes sense only in the many-query scenario

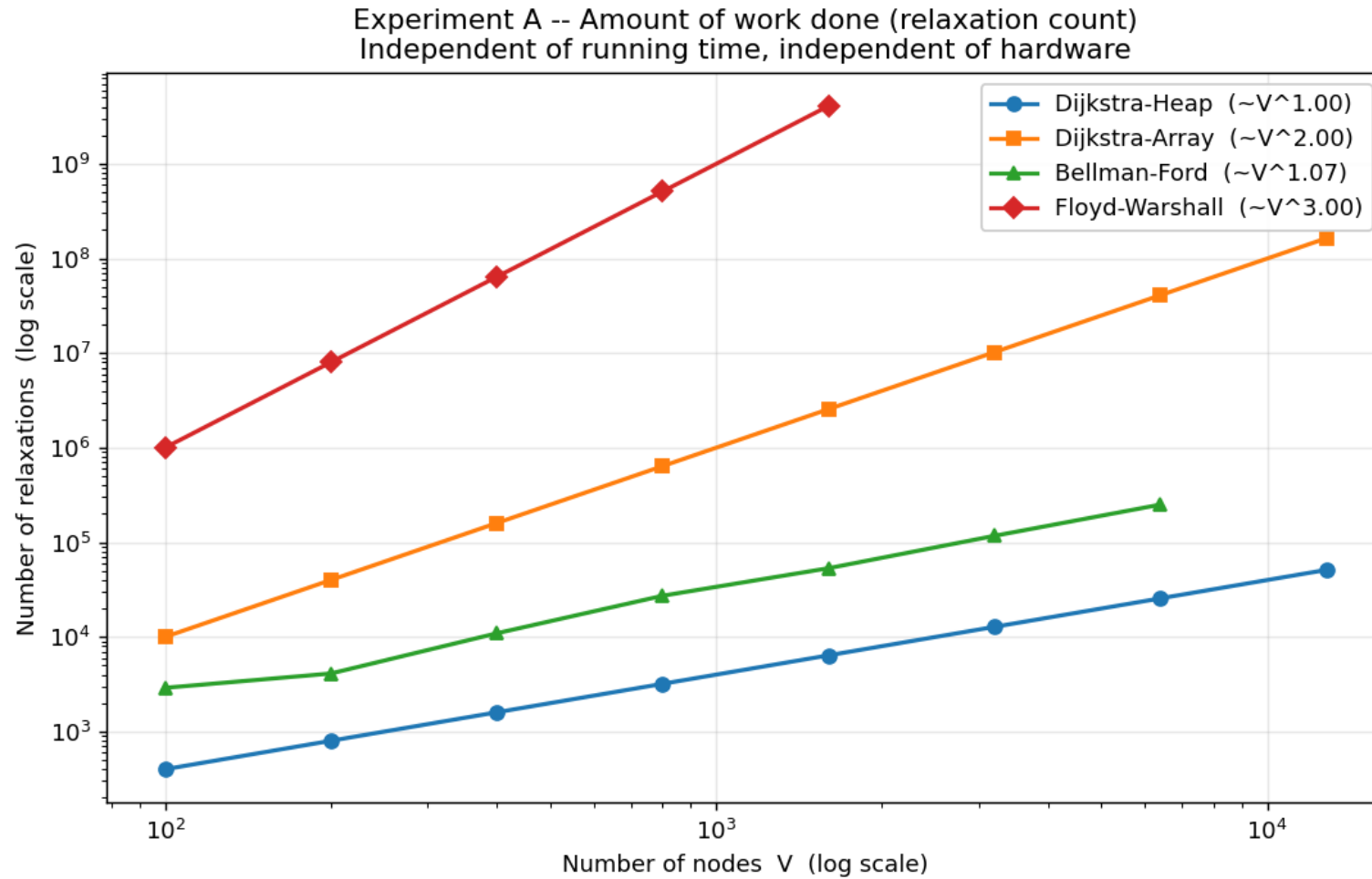
Experiment D — Real Road Network (DIMACS New York)



Takeaway

- 264,346 nodes, 733,846 arcs — real road distances
- Dijkstra-Heap answers an SSSP over the whole network in 802.8 ms, with 13 MB
- Adjacency matrix ~559 GB → Dijkstra-Array and Floyd-Warshall INFEASIBLE
- The representation choice can by itself determine feasibility

Comparing Theory with Experiment



Takeaway

- Operation-count (relaxation) exponents match theory EXACTLY: 2.00 / 3.00 / ≈ 1
- Hardware- and language-independent verification
- Time exponents deviate: NumPy overhead (small V) and memory bandwidth (large V^2)
- Theory gives the ordering; hardware constants shape it

Which Approach, When?

Scenario	Recommended	Why
Large, sparse graph; few queries	Dijkstra + Binary Heap	$O((V+E) \log V)$, low memory
Small/medium, very dense graph	Dijkstra + Array Scan	$O(V^2) \leq O(E \log V)$
Many queries; the graph is fixed	Floyd-Warshall	$O(V^3)$ preprocessing amortizes
Negative weights / cycle	Bellman-Ford	Dijkstra becomes incorrect; correctness is essential
Unweighted graph	BFS	$O(V+E)$; weighted ones are needlessly expensive

Takeaway

- Trade-offs: preprocessing vs. query · memory vs. access speed · speed vs. generality · theory vs. empirical constants

Conclusion

- ▶ **All four approaches verified** (cross-checks, hand examples, path reconstruction)
- ▶ **General-purpose, scalable option** Dijkstra + Binary Heap — the only practical option on the 264-thousand-node real road network (802.8 ms/query)
- ▶ **The "best" depends on the input** Dijkstra-Array if dense, Floyd-Warshall if many queries, Bellman-Ford if negative
- ▶ **The representation choice is decisive** $O(V^2)$ memory makes matrix-based approaches infeasible on large sparse graphs (~559 GB)
- ▶ **Theory vs. experiment** operation-count exponents match theory exactly; time exponents are shaped by constant factors/hardware

Takeaway

- Complexity analysis gives the DIRECTION of the decision; experimental measurement gives the CONSTANTS, the crossover points, and the feasibility limits.

Thank You

Questions and discussion